TASK 5 - Full Workflow

In this notebook, we have collected and organized the functions developed in Tasks 1–4 into separate sections. The functions are reused without rewriting their original logic, with minor adjustments to improve compatibility and readability.

At the end of the notebook, we combine these functions into a complete workflow that reads and processes transaction logs, cleans the data, generates summary reports, and saves the required output files.

In [1]:
```python
from pathlib import Path
import json
import re

import numpy as np
import pandas as pd
```

TASK 2 - RECRUSIVE UTILITY FUNCTIONS

In [2]:
```python
# Task 2.1 - Recursive Directory Search

def find_log_files(folder: Path) -> list[Path]:
    """
    Finner alle loggfilene i en mappe og undermappene.

    Args:
        folder (Path): Mappen som skal undersøkes.

    Returns:
        list[Path]: En liste med filbaner til loggfilene.
    """

    folder = Path(folder)
    log_files = []

    # Går gjennom alle elementene i mappen
    for item in folder.iterdir():

        # Hvis elementet er en mappe, undersøkes den rekursivt
        if item.is_dir():
            log_files.extend(find_log_files(item))

        # Hvis elementet er en .log-fil, legges den til i listen
        elif item.is_file() and item.suffix == ".log":
            log_files.append(item)

    return log_files


# Task 2.2 - Recursive JSON Traversal

def find_active_registers(data: object) -> set[str]:
    """
    Finner ID-ene til aktive kasseregistre i en datastruktur.
```

```
    Funksjonen undersøker dictionaries og lister rekursivt.

    Args:
        data (object): Dataene som skal undersøkes.

    Returns:
        set[str]: Et sett med ID-er til aktive kasseregistre.
    """

    active_registers = set()

    # Hvis data er en dictionary, undersøkes innholdet
    if isinstance(data, dict):

        # Legger til ID-en dersom kasseregisteret er aktivt
        if data.get("active") is True and "id" in data:
            active_registers.add(data["id"])

        # Undersøker alle verdiene i dictionaryen rekursivt
        for value in data.values():
            active_registers.update(find_active_registers(value))

    # Hvis data er en liste, undersøkes hvert element
    elif isinstance(data, list):
        for item in data:
            active_registers.update(find_active_registers(item))

    return active_registers
```

TASK 1 - FILE READING AND TRANSACTION PARSING

In [3]:

```
# Task 1.1

def fileread(filepath: str | Path) -> tuple[str, str]:
    """
    Read a text file using UTF-8 or Latin-1 encoding.

    Args:
        filepath (str | Path): Path to the file.

    Returns:
        tuple[str, str]: File content and the encoding used.
    """

    path = Path(filepath)

    # Try reading the file with UTF-8 encoding first
    try:
        with open(path, "r", encoding="utf-8") as file:
            content = file.read()
            encoding_used = "utf-8"

    # If UTF-8 fails, try Latin-1 encoding instead
    except UnicodeDecodeError:
        with open(path, "r", encoding="latin-1") as file:
            content = file.read()
            encoding_used = "latin-1"

    # Return both the file content and the encoding used
```

```
    return content, encoding_used

# Task 1.2

def parse_transactions(content: str) -> list[dict]:
    """
    Extract transactions from log file content using regex.

    Args:
        content (str): Text content from a log file.

    Returns:
        list[dict]: List of dictionaries containing transaction data.
    """

    # Create an empty list to store the transactions
    transactions = []

    # Go through the log file one line at a time
    for line in content.splitlines():

        # Search for transaction data using six capture groups
        match = re.search(
            r"\[(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\]\s\[(\w+)\]\sitem=(\w+)\s
            line
        )

        # Only process lines that match the transaction pattern
        if match:

            # Store the extracted values in a dictionary
            values = {
                "timestamp": match.group(1),
                "register_id": match.group(2),
                "item": match.group(3),
                "price": float(match.group(4)),
                "currency": match.group(5),
                "status": match.group(6)
            }

            # Add the transaction dictionary to the list
            transactions.append(values)

    # Return all extracted transactions
    return transactions
```

TASK 3 - NUMPY DATA PROCESSING

In [4]:

```
# Task 3.1 - Outlier Detection and Correction

def correct_outliers(prices: np.ndarray) -> np.ndarray:
    """
    Finner priser som avviker mer enn tre standardavvik fra gjennomsnittet
    og erstatter dem med medianen av de gyldige prisene.

    Args:
        prices (np.ndarray): En array med priser.

    Returns:
```

```
        np.ndarray: En array med korrigerte priser.
    """

    prices = np.array(prices, dtype=float, copy=True)

    # Returnerer en tom array dersom ingen priser er oppgitt
    if prices.size == 0:
        return prices

    # Beregner gjennomsnittet og standardavviket
    mean = np.mean(prices)
    std = np.std(prices)

    # Finner priser som ligger mer enn tre standardavvik fra gjennomsnittet
    outliers = np.abs(prices - mean) > (3 * std)

    # Finner prisene som ikke er outliers
    valid_prices = prices[~outliers]

    # Beregner medianen av de gyldige prisene
    median = np.median(valid_prices)

    # Erstatter outliers med medianen
    prices[outliers] = median

    return prices


# Task 3.2 - Refund Ratio

def compute_refund_ratio(
    sales_count: np.ndarray,
    refund_count: np.ndarray
) -> np.ndarray:
    """
    Calculate the element-wise refund ratio using NumPy arrays.
    Adds a small epsilon to prevent division by zero.
    Returns an array containing the calculated refund ratios.
    """

    # Small value to prevent division by zero
    epsilon = 1e-6

    # Calculate refund ratio for each element
    ratio = refund_count / (sales_count + epsilon)

    return ratio
```

TASK 4 - PANDAS DATA ANALYSIS

In [5]:

```
# Task 4.1 - Create DataFrame

def create_dataframe(transactions: list[dict]) -> pd.DataFrame:
    """
    Gjør en liste med transaksjoner om til en DataFrame.

    Args:
        transactions (list[dict]): Liste med transaksjonsdata.
```

```
    Returns:
        pd.DataFrame: Transaksjoner sortert etter tidspunkt.
    """

    # Lager en DataFrame fra transaksjonslisten
    df = pd.DataFrame(transactions)

    # Gjør tidspunktet om til datetime-format
    df["timestamp"] = pd.to_datetime(df["timestamp"])

    # Bruker tidspunktet som indeks
    df = df.set_index("timestamp")

    # Sorterer transaksjonene kronologisk
    df = df.sort_index()

    return df


# Task 4.2 - Hourly Revenue

def hourly_revenue(df: pd.DataFrame) -> pd.DataFrame:
    """
    Beregner summen av transaksjonspriser per time og produkt.

    Args:
        df (pd.DataFrame): DataFrame med transaksjoner.

    Returns:
        pd.DataFrame: Summert pris per time og produkt.
    """

    # Grupperer etter produkt og summerer prisene per time
    hourly = df.groupby("item").resample("1h")["price"].sum()

    # Gjør resultatet om til en DataFrame
    return hourly.to_frame(name="price")


# Task 4.3 - Transaction Pivot Table

def transaction_pivot(df: pd.DataFrame) -> pd.DataFrame:
    """
    Create a pivot table showing total transaction revenue per item and status.
    Returns a DataFrame with items as rows and transaction statuses as columns.
    """

    # Group transactions by item and status, summing the prices
    pivot = pd.pivot_table(
        df,
        index="item",
        columns="status",
        values="price",
        aggfunc="sum",
        fill_value=0
    )

    return pivot
```

```python
# Task 4.4 - Refund Ratio per Item

def refund_ratio_per_item(df: pd.DataFrame) -> pd.DataFrame:
    """
    Calculate the refund ratio per item using the function from Task 3.2.
    Returns a DataFrame containing the refund ratio for each item.
    """

    # Count transactions for each item and status
    counts = pd.crosstab(df["item"], df["status"])

    # Get the number of successful sales and refunds per item
    sales_count = counts.get("SUCCESS", pd.Series(0, index=counts.index))
    refund_count = counts.get("REFUND", pd.Series(0, index=counts.index))

    # Calculate refund ratios using the function from Task 3.2
    ratios = compute_refund_ratio(
        sales_count.to_numpy(),
        refund_count.to_numpy()
    )

    # Return a DataFrame with item names and refund ratios
    return pd.DataFrame({"refund_ratio": ratios}, index=counts.index)
```

TASK 5 - WORKFLOW IMPLEMENTATION

1. Recursively find all log files and extract active register IDs

In [14]:
```python
# Find all log files in the data folder
log_files = find_log_files(Path("data"))

# Print the file paths
for file in log_files:
    print(file)
```

```
data\2026\library_cafe\POS_01\register_sales.log
data\2026\library_cafe\POS_02\register_sales.log
data\2026\library_cafe\POS_03\register_sales.log
data\2026\science_building\POS_01\register_sales.log
data\2026\science_building\POS_02\register_sales.log
data\2026\science_building\POS_03\register_sales.log
data\2026\student_union\POS_01\register_sales.log
data\2026\student_union\POS_02\register_sales.log
data\2026\student_union\POS_03\register_sales.log
```

In [15]:
```python
# Read the JSON configuration file
with open("data/store_config.json", "r", encoding="utf-8") as file:
    config = json.load(file)

# Find all active register IDs
active_registers = find_active_registers(config)

# Print the active register IDs
print("Active registers:", sorted(active_registers))
```

```
Active registers: ['POS_01', 'POS_02', 'POS_03']
```

2. Ingest text logs and parse records into a list of dictionaries

In [8]:
```
# Create an empty list to collect transactions from all log files
all_transactions = []

# Go through each log file found in Task 2
for filepath in log_files:

    # Read the file content using the function from Task 1
    content, encoding_used = fileread(filepath)

    # Extract transactions from the file content
    transactions = parse_transactions(content)

    # Add the transactions to the combined list
    all_transactions.extend(transactions)

# Print the total number of transactions
print("Total transactions:", len(all_transactions))

# Show the first three transactions
for transaction in all_transactions[:3]:
    print(transaction)
```

```
Total transactions: 720
{'timestamp': '2026-10-15 07:01:46', 'register_id': 'POS_01', 'item': 'Muffin',
'price': 3.53, 'currency': 'NOK', 'status': 'CANCELLED'}
{'timestamp': '2026-10-15 07:02:23', 'register_id': 'POS_01', 'item': 'Croissan
t', 'price': 3.4, 'currency': 'USD', 'status': 'SUCCESS'}
{'timestamp': '2026-10-15 07:03:38', 'register_id': 'POS_01', 'item': 'Cappuccin
o', 'price': 4.19, 'currency': 'EUR', 'status': 'SUCCESS'}
```

3. Build the initial Pandas DataFrame

In [9]:
```
# Create a DataFrame from all parsed transactions
df = create_dataframe(all_transactions)

# Print the number of rows and columns
print("DataFrame shape:", df.shape)

# Display the first five transactions
display(df.head())
```

```
DataFrame shape: (720, 5)
```

timestamp	register_id	item	price	currency	status
2026-10-15 07:00:49	POS_01	Croissant	4.14	EUR	REFUND
2026-10-15 07:00:51	POS_03	Muffin	999.00	NOK	REFUND
2026-10-15 07:00:59	POS_03	Croissant	3.46	USD	SUCCESS
2026-10-15 07:01:16	POS_03	Cappuccino	4.01	USD	SUCCESS
2026-10-15 07:01:30	POS_02	Muffin	3.31	NOK	SUCCESS

4. Apply NumPy outlier cleaning to remove 999.00 price glitches

In [10]:
```
# Apply NumPy outlier cleaning to the price column
df["price"] = correct_outliers(df["price"].to_numpy())
```

5. Apply currency correction for NOK transactions

In [11]:
```
# Correct NOK prices by multiplying them by 10
df.loc[df["currency"] == "NOK", "price"] *= 10.0

# Display the first five transactions after cleaning and correction
display(df.head())
```

timestamp	register_id	item	price	currency	status
2026-10-15 07:00:49	POS_01	Croissant	4.14	EUR	REFUND
2026-10-15 07:00:51	POS_03	Muffin	40.00	NOK	REFUND
2026-10-15 07:00:59	POS_03	Croissant	3.46	USD	SUCCESS
2026-10-15 07:01:16	POS_03	Cappuccino	4.01	USD	SUCCESS
2026-10-15 07:01:30	POS_02	Muffin	33.10	NOK	SUCCESS

6. Generate resampled reports, pivot tables and refund ratios

In [12]:
```
# Generate reports using the functions from Task 4
hourly_report = hourly_revenue(df)
pivot_report = transaction_pivot(df)
refund_report = refund_ratio_per_item(df)

# Display the generated reports
print("Hourly revenue report:")
display(hourly_report.head())

print("Transaction pivot table:")
display(pivot_report)
```

```
print("Refund ratios per item:")
display(refund_report)
```

```
Hourly revenue report:
```

		price
item	**timestamp**	
Cappuccino	**2026-10-15 07:00:00**	772.18
	2026-10-15 08:00:00	1034.42
	2026-10-15 09:00:00	157.57
Croissant	**2026-10-15 07:00:00**	1035.10
	2026-10-15 08:00:00	993.32

```
Transaction pivot table:
```

status	**CANCELLED**	**REFUND**	**SUCCESS**
item			
Cappuccino	382.45	228.02	1353.70
Croissant	452.67	295.91	1374.09
Espresso	234.61	206.96	790.81
Latte	336.23	214.48	1482.73
Muffin	321.85	481.78	1105.75
Sandwich	240.27	1075.83	1882.78

```
Refund ratios per item:
```

	refund_ratio
item	
Cappuccino	0.207792
Croissant	0.258065
Espresso	0.234568
Latte	0.266667
Muffin	0.311688
Sandwich	0.369863

7. Save all required output files

In [13]:
```
# Create an output folder if it does not already exist
output_folder = Path("output")
output_folder.mkdir(exist_ok=True)

# Save the cleaned transactions as CSV
df.to_csv(output_folder / "cleaned_transactions.csv")
```

```
# Save the hourly revenue report as JSON
hourly_report.reset_index().to_json(
    output_folder / "hourly_revenue.json",
    orient="records",
    date_format="iso",
    indent=4
)

# Save the status pivot table as CSV
pivot_report.to_csv(output_folder / "status_pivot_table.csv")

# Save the refund ratios as CSV
refund_report.to_csv(output_folder / "item_refund_ratios.csv")

# Confirm that all output files were saved
print("All four output files have been saved in the output folder.")
```

```
All four output files have been saved in the output folder.
```

Results and Discussion

The workflow identified 9 log files and extracted 720 transactions from the available data. The transactions were organized into a DataFrame, and the price cleaning process corrected unusually high values before adjusting NOK prices. The hourly revenue report summarizes transaction values by item and hour, while the pivot table compares transaction values across different statuses. The refund ratio report shows the relationship between refunds and successful sales for each item. Together, these reports provide an overview of the processed transaction data.

Conclusion

This task demonstrates how the functions developed in Tasks 1–4 can be combined into a complete data processing workflow. By automating the process, transaction data can be collected, cleaned, analyzed, and exported in a structured and reusable way. The workflow provides a foundation for further analysis of the coffee shop's sales data.